



SWE30003
Software Architectures and Design

School of SCET
Semester 2, 2025

Unit Organisation

1

Unit Teaching Staff



- **Convenor/Lecturer:**
Prof Jun Han
Email: jhan@swin.edu.au
Office: EN 507b
Phone: (03) 9214 5732
Consultation: 2.30pm~3.15pm Wednesday (Canvas/Collaborate)
- **Tutors:**
Mr Mohammad Abuhassan, mabuhassan@swin.edu.au
Dr Naveed Ali, Tutor, nali1@swin.edu.au.
Ms Mandeep Dhindsa, Tutor, mdhindsa@swin.edu.au.
Ms Thakshila Dilrukshi, Tutor, timiyamohottige@swin.edu.au.
Dr Mostafa Farshchi, Tutor, mfarshchi@swin.edu.au.
(further info on Canvas)

2

2

Aims of this Unit of Study



This unit aims to facilitate an in-depth study of state-of-the-art approaches and techniques for software system design with a special focus on the relationship between non-functional requirements and software architectures.

3

3

Goals of this Unit of Study



- To illustrate the importance of requirements analysis and appropriate notations for requirements documentation,
- To illustrate the relationship between requirements analysis and software design,
- To discuss and compare design strategies and approaches,
- To give students hands-on experience with current approaches and techniques in system design,
- To highlight the importance of verification and validation at various stages of the software development lifecycle.
- To enhance students' ability to read, understand, and discuss academic/scientific publications.

4

4

Organization



- Each lecture session covers a particular topic as indicated in the schedule.
- Lectures will be used to *highlight* the most important issues of each topic.
- Students are expected to read a given book chapter/research paper *prior* to each lecture session. For further study, complementary readings will also be given.
- Students are required to submit one question related to the weekly pre-reading as well as an answer for a given question for the topic covered in the past week, on a weekly basis.
- A selection of submitted questions will be discussed in some lectures/tutorials (time permitting) - some will be used for the final assessment test.
- Tutorial classes will be organized for further discussion of selected topics/questions and give detailed feedback on assignments.

5

5

Provisional Schedule



Week	Lecture	assessment due
1	Unit Overview; Issues in Software Design	
2	Goal-Design Scale, User Tasks	Weekly Q&A
3	Quality Attributes, Requirements Validation	Weekly Q&A
4	Domain Modelling, Software Abstractions	Weekly Q&A
5	Responsibility-Driven Design	Weekly Q&A
6	Detailed Object Design	Weekly Q&A, Requirements Specification
7	Design Patterns	Weekly Q&A
8	Software Architectures, Architectural Styles	Weekly Q&A
9	Case Study in Architectural Design	Weekly Q&A, Object-Oriented Design I
10	Service Oriented Architecture and Web Services	Weekly Q&A
11	Documenting Designs	Weekly Q&A
12	Design Evaluation; Wrapping up	Object-Oriented Design II
FAP		Final Assessment (Test/Exam) – in-person, invigilated

6

6

Lectures and Tutorials



- All **lectures** are online:
Canvas / Collaborate Ultra – live (not prerecorded)
- **Tutorials** are face-to-face on campus

7

7

Recording of Lectures



- All lectures *may* be recorded
- Recordings will be made available through Canvass *as is* – no guarantees given about the quality of the recordings (or lack thereof).
- Note: recordings are not a replacement for regularly attending classes!

8

8

Assessment



- Assignment 1 (Requirements; due week 6) 15%
- Assignment 2 (Object Design I; due week 9) 20%
- Assignment 3 (Object Design II; due week 12) 15%
- Weekly Questions and Snswers (Canvas) 10%
(submission closes **Tuesday 08h00am**)
- Final Assessment (Test/Exam)(invigilated, details TBA) 40%

NO genAI is to be used in any assessment submissions!

9

9

Assessment (cont.)



- Weekly readings: essential normal study, not direct part of assessment (weekly Q&A)- but related
- Weekly Q&A: Submission of an unsuitable question/answer or late/no submission – 0 mark.
- A penalty of 1% may apply for each question copied from (i) one of the given textbooks or articles or AI tools and or (ii) from another student/source! ... should be your understanding and wording
- To pass this subject, an **overall mark of 50%, 40% or more** in the final assessent, AND **at least 40%** for the weekly Q&A submissions!
- ☞ Assessment criteria available on Canvas (unit outline and assignment specifications).

10

10

Final Assessment (Test/Exam)



- The final test will be **2~3 hours**, In final assessment period.
- The precise date of the test will be confirmed by university in due course.
- Testable material is **everything** that was talked about either in the lectures or tutorials, covered in the three assignments, discussed in the weekly questions/answers submission, or the readings.
 - ☞ *This explicitly includes material that may not be part of the lecture or tutorial notes!*
- For various reasons, predominantly educational nature, there will be **no sample test** made available - hence please do not ask for one. However, **some sample questions** that were used in the past will be provided and discussed in Week 12. See also questions in lectures and tutorials.
- Please post messages to the discussion forum for any issues in₁₁ regards to the final test, when the time comes.

11

Assignments – Group Formation



- All three assignments in this Unit of Study are to be completed in **teams of the same 3 or 4 students**.
- **All members of a group must be from the same tutorial class.**
- A “Group Form Sheet” is available on Canvas (under Week 1) that is to be completed by all teams and returned to the tutor asap in week 1 tutorial, but no later than **Friday, week 2**.
 - ☞ *Please do not forget to nominate a contact person, and keep a copy for yourself!*
- There is a dedicated discussion board on Canvas that can be used to form teams and/or find team members (when outside tutorial session).
 - ☞ If you cannot find a team, contact the tutor and you will be allocated to an existing team (if possible).
 - ☞ If you *do not* contact the tutor before the group formation deadline, you will have to find your own team.

12

12

Principal References



- Soren Lauesen, *Software Requirements: Styles and Techniques*, Addison-Wesley, 2002
 - Len Bass, Paul Clements, and Rick Kazman, *Software Architecture in Practice (4th Edition)*, Pearson, 2021.
 - David Budgen, *Software Design (2nd Edition)*, Addison-Wesley, 2003
 - Eric Evans, *Domain-Driven Design*, Addison-Wesley, 2004
 - Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides, *Design Patterns*, Addison-Wesley, 1995
 - Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad and Michael Stal, *Pattern-Oriented Software Architecture: A System of Patterns*, Wiley 1996
 - Ian Sommerville, *Software Engineering (Global Edition)*, Pearson, 2016
 - Jeff Garland and Richard Anthony, *Large-Scale Software Architecture*, Wiley, 2003
 - Rebecca Wirfs-Brock and Alan McKean, *Object Design*, Addison-Wesley, 2003
- A few more references will be given during the semester!

13

13

Note...



- The emphasis of this unit is mostly on *pre-implementation* aspects of software development (assuming you are competent in software programming/implementation).
- We will re-cover some of the ground from OOP, but with a different perspective, and in some lectures you may get a sense of “déjà vu” – and that is OK, as it is important to recall the context.
- But, the same concept may have a more advanced meaning/explanation in the context of this unit ... (OOD != OOP)
- It also gives us the opportunity to revisit questions whose answers you should know (since you passed OOP...). 14

14

Expectation for Students



15

15

Some important context / expectations (1)



This unit is different from most prior/other units:

- “Software requirements, & software design” ...
- not black-white answer, or simple right-or-wrong ...
- It is about making the right judgement/decision ...
subjective, but rationalised
- Assignment specifications may appear “open”, or “unclear”:
 - ☐ Meant to be non-prescriptive (reqs & design),
 - ☐ need rational assumptions & answers,
 - ☐ Should be based on unit knowledge and judgement

16

16

Some important context / expectations (2)



This unit is different from most prior/other units:

- Lectures are REALLY just highlights (not everything), but need to get full understanding & nuances from reading & reflection
- Essential **self-reading, self-driven** ... beyond lectures/tutorials
- **Reading & reflection** may appear “boring”, but, **essential** skills to build ... can be “exciting” when you get it! ... library search too
- Some reading materials may be “old”, but classics.
- Tutorials:
 - ☐ **come prepared** – try to answer the questions prior,
 - ☐ May need to complete afterward
- Post-lecture self-notes ...

17

17

Some important context / expectations (3)



This unit is different from most prior/other units:

- **Concepts** may be familiar, but different/richer

- Eg,

OOD != OOP
Objects in OOD != objects in OOP
(some are not!!)

18

18

Some important context / expectations (4)



This unit is different from most prior/other units:

- “Contexts” are important ...
- **Unit context**: particular methods, techniques, approaches, perspectives of this unit need to follow (even there are others)!
Eg, use case driven RE vs. task&support driven RE
- **Problem context**: a particular method or approach may not be appropriate for a given problem.
Eg, OOD vs microservices

19

19

Some important context / expectations (5)



Do question everything, but follow what this unit is about:

- **Industrial experience & practice** are useful,
but perspective may be limited / wrong / inappropriate (in a given context)!
- Accepting **limitations** in understanding and knowledge leads to in-depth learning
- Eg, “the three assignments ...”
- Methodical software engineering vs “hacking” (bad practices)...

20

20

Some important context / expectations (6)



True group/team work is essential:

- Industrial practice
- Collaboration leads to good/better outcomes – reviewed, consistent ...
- In particular, in rationalised (design) decision-making (not just the first thought about)
- Team submission should NOT be a simple “assembly” of individuals’ work – un-refined, inconsistent, not to mention sub-optimal
- A student comment: “bad assignments as we can not divide the work cleanly ...” --- not bad assignment, but bad response! ²¹

21

Quotes (1)



“Although i missed a few, asking questions from weekly readings really forced me to read literature from the broader context of the field. The talk about developing a language was very interesting and i'm very glad i was forced to watch it. You never know what you don't know and i would rarely consume this literature on my own time.”

22

22

Quotes (2)



“The topics. This class has been wonderfully helpful in showing how a system is designed; a step that has been severely lacking in BACS up to now. The three assignments providing the step by step experience of designing and implementing a system also makes me a lot more confident and comfortable going into my capstone units next year.”

23

23



So you know what to do ... 😊

24

24